Reinforcement Learning on ALE/KungFuMaster

A Comparative Study of DQN and PPO

Andrea Zanin Jad Mahmoud Mahammed Hasen Muhummed

University of Pisa
Department of Information Engineering
M.Sc. in Artificial Intelligence and Data Engineering

Symbolic and Evolutionary AI — A.Y. 2025/2026
Instructor: Prof. Marco Cococcioni

June 2026

Outline

Introduction and Motivation

Project Goal

Compare two foundational families of deep RL algorithms on a challenging Atari benchmark to understand their relative strengths, sample efficiency, and behavior under custom reward shaping.

Why DQN vs PPO?

- **DQN** (Mnih et al., 2015): off-policy, value-based.
- **PPO** (Schulman et al., 2017): on-policy, policy-gradient with clipped surrogate.
- Two dominant deep-RL paradigms.

Why ALE/KungFuMaster-v5?

- Sparse, deceptive rewards (reward-hacking risk).
- Long-horizon credit assignment across hundreds of steps.
- Visual complexity (fast projectiles, small sprites).
- Genuinely differentiates the algorithms.

Research Question

Given identical preprocessing and matched training budgets, do DQN and PPO achieve statistically equivalent performance, or does one paradigm provide a clear advantage?

The Environment: ALE/KungFuMaster-v5

Atari 2600 beat-em-up exposed through the Arcade Learning Environment (ALE) and the Gymnasium API.

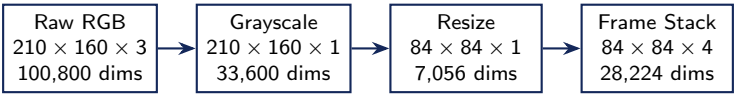
Game Overview

- **Objective:** fight through the Devil's Temple to rescue Sylvia from Mr. X.
- **Enemies:** Grippers, Knife Throwers, Tom Toms, snakes, dragons, falling balls.
- **Scoring:**
  - Standard enemy: $+100$ to $+200$
  - Boss defeat: $+2000$
- **Lives:** 3 per evaluation episode; we terminate after the first death during training.

Technical Specifications

- **Observation:** $210 \times 160 \times 3$ RGB
- **Action space:** `Discrete(14)`
- **Sticky actions:** default 0.25, we set to 0.0
- **Frame skip:** 4, max-pool last 2

MDP Formulation: State Space



Nature of the State

- High-dimensional pixel input (`uint8`, effectively continuous for learning).
- Same protocol as Mnih et al. (2015) — standard Atari practice.
- `uint8` kept for memory efficiency ($\sim$4$\times$ savings vs `float32`); converted to float and normalized inside the network.

Frame Stacking

- A single frame is non-Markovian: velocity and projectile direction are hidden.
- Stacking the last 4 frames **restores the Markov property** via temporal differences.
- Cost: 4$\times$ memory and compute per forward pass.

MDP Formulation: Action Space

`Discrete(14)` — one of 14 actions per timestep

The 14 actions

ID	Action	ID	Action
0	NOOP	7	RIGHTFIRE
1	UP	8	LEFTFIRE
2	RIGHT	9	DOWNFIRE
3	LEFT	10	UPRIGHTFIRE
4	DOWN	11	UPLEFTFIRE
5	DOWNRIGHT	12	DOWNRIGHTFIRE
6	DOWNLEFT	13	DOWNLEFTFIRE

Implications for Algorithm Choice

- Discrete action space $\Rightarrow$ $Q(s, a)$ has finite outputs $\Rightarrow$ value-based methods (DQN) directly applicable via $\pi(s) = \arg\max_a Q(s, a)$.
- **Both algorithms valid here:**
  - DQN: requires discrete actions
  - PPO: handles both discrete and continuous
- If actions were continuous, DQN would not apply directly — DDPG/SAC/TD3 would be required.

A discrete action space lets us compare value-based and policy-based methods without confounding from action-encoding differences.

Reward Structure and Stochasticity

Reward Function (sparse, event-driven)

Event	Reward
Standard enemy (Tom Tom, Knife Thrower)	+100
Tougher enemy (Gripper)	+200
Floor boss defeat	+2000
Damage / movement / idle	0

Stochasticity Sources

Source	Default	Ours	Rationale
`repeat_action_probability`	0.25	0.0	Cleaner comparison; less noise
`noop_max` on reset	30	30	Initial-state diversity
Enemy spawn timing	semi-det.	semi-det.	Inherent ALE behavior

Reward Hacking Risks

- **Minion farming:** waves of 3 enemies spawn endlessly (+600/wave for trivial behavior).
- **Crouching defense:** corner agent kicks attackers, never advances.
- **Score $\neq$ progress:** running forward + boss kill earns $\sim$2800–3500. A passive minion-farmer can score much higher *without solving the level.*

Mitigation: custom reward shaping (Slide 13) explicitly rewards milestone progression.

DQN: Mathematical Foundations

1. The Q-Function (Action-Value)

$$Q^{\pi}(s,a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \,\middle|\, s_0 = s, a_0 = a, \pi\right]$$

2. Bellman Optimality Equation

$$Q^{*}(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^{*}(s',a') \,\middle|\, s,a\right]$$

3. Deep Q-Learning Update

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y - Q_\theta(s,a)\right)^2\right], \quad y = r + \gamma \max_{a'} Q_{\theta^-}(s',a')$$

$\mathcal{D}$: replay buffer. θ^-: slowly-updated target network. We use **Huber loss** for robustness.

Key Properties

- **Off-policy** — Learns from any past experience $\Rightarrow$ enables replay buffer.
- **Value-based** — Policy is implicit: $\pi(s) = \arg\max_a Q_\theta(s,a)$.
- **Bootstrapping** — Targets depend on Q_{θ^-} itself $\Rightarrow$ requires stabilizers.

DQN: Architecture and Implementation

Network Architecture (mirrors Mnih et al., 2015)

Layer	Type	Filters/Units	Kernel	Stride	Activation
Conv1	2D Conv	32	8×8	4	ReLU
Conv2	2D Conv	64	4×4	2	ReLU
Conv3	2D Conv	64	3×3	1	ReLU
FC1	Linear	512	—	—	ReLU
FC2 (output)	Linear	14	—	—	Linear (Q-values)

Stability Mechanisms

- **Target network** Q_{θ^-}, hard sync every **2,500 steps**.
- **Custom circular replay buffer:** stores single 84×84 frames (not 4-frame tensors); 4-frame stacks reconstructed *on-the-fly* from buffer indices.
- 10% extra slots pre-allocated; transitions whose frames get overwritten are popped. 3M-transition buffer occupies ∼21 GB RAM.
- **Double DQN**; uniform sampling; batch size 128.

Two-Phase ε-Decay

- Phase 1: $1.0 \rightarrow 0.10$ over 2M steps (broad exploration).
- Phase 2: $0.10 \rightarrow 0.01$ over 8M steps (fine-tuning).
- Held at 0.01 thereafter.

Training Hyperparameters

- Adam, $\eta = 1\text{e}{-4}$, $\gamma = 0.99$.
- Huber loss; train every 4 steps; warm-up 80k steps.
- Configured: 15M timesteps; actual run: 3M (stopped for time budget).

PPO: Mathematical Foundations

1. The Policy and Objective

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \gamma^t r_t\right], \quad \nabla_\theta J(\theta) = \mathbb{E}_{(s,a)\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\cdot \hat{A}^\pi(s,a)\right]$$

Advantage: $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$.

2. Clipped Surrogate Objective

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\right)\right]$$

Clip range $\varepsilon = 0.1$. The `min` creates a *pessimistic* lower bound preventing destructive updates.

3. Total Loss (Actor-Critic)

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 S[\pi_\theta]$$

L^{VF}: value MSE. $S[\pi_\theta]$: entropy bonus ($c_1 = 0.5$, $c_2 = 0.01$). GAE $\lambda = 0.95$ (SB3 default).
Key Properties: On-policy, policy-based, clipped trust-region (no expensive KL).

PPO: Architecture and Implementation

Network Architecture (shared CNN trunk + two heads, via SB3 `CnnPolicy`)

Layer	Type	Filters/Units	Kernel	Stride	Activation
Conv1	2D Conv	32	8×8	4	ReLU
Conv2	2D Conv	64	4×4	2	ReLU
Conv3	2D Conv	64	3×3	1	ReLU
Shared FC	Linear	512	—	—	ReLU
Policy head	Linear	14	—	—	Softmax
Value head	Linear	1	—	—	Linear

On-Policy Rollouts

- `SubprocVecEnv` $\times$ **8 envs** (process-level parallelism, different per-env seeds).
- Rollout: $128 \times 8 = 1024$ steps/iteration.
- 4 epochs per rollout, batch size 256.
- **No replay buffer.**

Added Engineering Beyond SB3

- `TrueScoreWrapper`: raw game score tracking.
- PPO-specific reward shaping (Slide 14).
- Custom CSV callback compatible with DQN's schema.
- Resume checkpointing every 100k steps.
- Periodic in-training video recording.

Hyperparameters: $\eta = 2.5\text{e}{-4}$, $\gamma = 0.99$, *clip* $\varepsilon = 0.1$, *ent_coef 0.01, vf_coef 0.5, max_grad_norm 0.5.*

Custom Preprocessing Pipeline

Standard Atari Preprocessing

- Random no-op reset $\in [1, 30]$ — anti-overfitting.
- Frame skip $= 4$, max-pool last 2 — recovers flickering sprites.
- Grayscale + resize to 84×84 (cv2 area interpolation).
- **UI mask**: zero rows 0–36 (score/timer) and 65+ (floor).
- Stack 4 processed frames as channels.

Knife Dilation (custom)

Problem: knife sprites are 1–2 px at 84×84 — invisible to the CNN.

Fix: identify by RGB (74,74,74), dilate with 3×3 kernel, paint white *before* downsizing.

```
mask = cv2.inRange(play_area,
                   knife_color,
                   knife_color)
dilated = cv2.dilate(mask, kernel)
play_area[dilated > 0] = [255, 255, 255]
```

Impact: knives survive downsampling — agent learns to dodge.

Reward Engineering: Custom Shaping

Base reward is sparse (Slide 7). We add a dense, multi-component signal computed from **RAM-derived game state**.

Shaped Reward Formula

$$r_{\text{shaped}} = r_{\text{step}} + \frac{r_{\text{raw}}}{\text{scale}} + r_{\text{milestone}} + r_{\text{boss\_dmg}} - r_{\text{health}} - r_{\text{death}} + r_{\text{clear}}$$

Eight Components (DQN values)

Component	Value
step_penalty	−0.005
scale_factor	1000.0
base_explore_reward	+0.5
milestone_bonus	+0.2
health_penalty	−0.1
death_penalty	−3.0
level_clear_reward	+25.0
boss_dmg_reward	+0.1

RAM Bytes Used

verified by manual play + memory injection (A. Zanin)

Byte	Meaning
6	Corridor position (0–12)
29	Lives remaining (3–0)
75	Player health (39–0)
76	Boss HP (39–0)

RAM for Reward, Not Observation

Agent input = pixels only.
RAM = reward-designer's tool, equivalent to any human-designed reward.

Why Reward Shaping Must Differ: DQN vs PPO

Component	DQN	PPO	Change
`step_penalty`	-0.005	0.0	**Disabled for PPO**
`milestone_bonus`	$+0.2$	0.0	Disabled for PPO
`health_penalty`	-0.1	-0.02	$5\times$ softer
`death_penalty`	-3.0	-1.0	$3\times$ softer
`level_clear_reward`	$+25.0$	$+5.0$	$5\times$ softer
`base_explore_reward`	$+0.5$	$+0.5$	Same
`boss_dmg_reward`	$+0.1$	$+0.1$	Same
`scale_factor`	1000	1000	Same

Empirical finding: `step_penalty` collapses PPO

With `step_penalty` $= -0.005$ (works perfectly for DQN), PPO's policy collapsed to `NOOP` — the agent learned to NOT MOVE within $\sim$300k steps.

DQN interprets `step_penalty` as:

- Uniform downward shift of all Q-values.
- $\arg\max_a Q$ unchanged $\Rightarrow$ behavior preserved.
- Mild urgency signal $\Rightarrow$ works as intended.

PPO interprets `step_penalty` as:

- All sampled actions get systematically negative $\hat{A}_t$.
- Policy gradient pushes *away* from every action.
- Entropy bonus cannot recover $\Rightarrow$ policy collapses.

Training Setup and Computational Resources

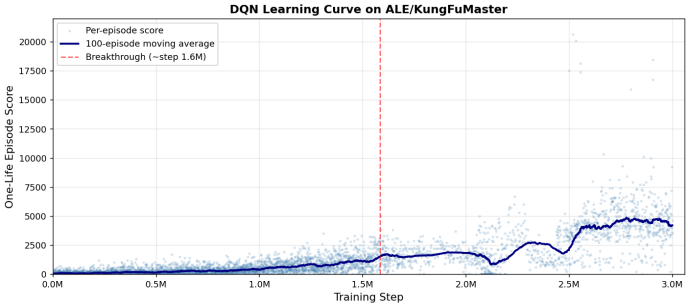
Hardware

Resource	Role
Local workstation (64 GB RAM)	DQN training (large replay buffer)
Google Colab Pro T4	PPO training, development, debugging

Parameter	**DQN**	**PPO**
Optimizer	Adam	Adam
Learning rate	1e−4	2.5e−4
Discount γ	0.99	0.99
Batch size	128	256
Loss	Huber	Clipped surrogate + value MSE + entropy
Grad clip (max-norm)	—	0.5
Replay buffer	3M (custom circular, ∼21 GB)	— (on-policy)
Target network sync	2,500 steps	—
Train frequency	every 4 steps	—
Warm-up (learning_starts)	80,000 steps	—
ε-decay	two-phase 1.0 → 0.10 (2M) → 0.01 (+8M)	—
Parallel envs / per-env seeds	1	8 (different seeds per env)
Rollout length	—	1024 (128×8)
Epochs per rollout	—	4
Clip range ε	—	0.1
GAE λ	—	0.95 (SB3 default)
Entropy coef	—	0.01
Value coef	—	0.5
Configured timesteps	15,000,000	15,000,000
Actual training	3,000,000 (stopped for time budget)	3,000,000 (same)

Same step budget ⇒ fair sample-efficiency comparison; PPO's wall-clock advantage comes from parallelism, not algorithm.

DQN Training Results



Key Statistics

Actual training steps	3M
Total episodes	5,667
Max training (1-life)	20,620
Mean eval score*	**21,916**
Std dev (eval)	5,429.60
Max eval score	**27,500**
Breakthrough step	$\sim$1.6M
ε at breakthrough	$\approx$0.28

**Eval = full 3-lives game; training terminated after first death.*

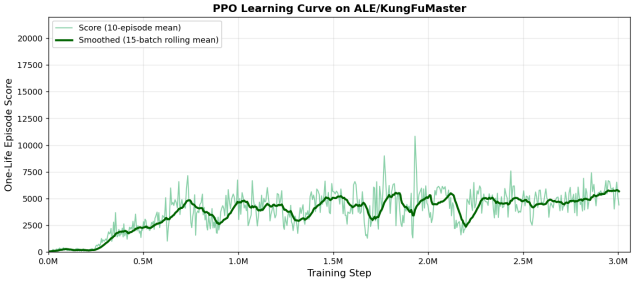
▶ **Watch DQN play**

Three Phases of Learning

Phase	Episodes	ε range	Raw score	Behavior
Random exploration	0–2000	1.0 $\rightarrow$ 0.76	0–300	Random actions; quick deaths
Slow learning	2000–4500	0.76 $\rightarrow$ 0.28	300–2000	Basic combat; occasional level clears
Competent play	4500+	0.28 $\rightarrow$ 0.09	2000–20,620	Reliably clears level 1

Breakthrough at step $\sim$1.6M: 100-episode rolling score climbs past 1,500 and continues upward to $\sim$5,000 by 3M.

PPO Training Results



Key Statistics

Actual training steps	3M
Parallel envs	8 (SubprocVecEnv)
Cumulative episodes	∼5,865
Max training (1-life)	10,826
Mean eval score*	**10,312**
Std dev (eval)	5,108.45
Max eval score	**19,000**

**Eval = full 3-lives game.*

▶ **Watch PPO play**

PPO Learning Trajectory

Phase	Approx. steps	Behavior
Initial exploration	0–300k	High-entropy stochastic policy; low scores
Rapid improvement	300k–1M	Commits to combat; one-life score climbs to ∼5,000
Plateau / refinement	1M–3M	Small oscillating gains; training stopped at 3M

Algorithm-specific observations

PPO's curve is smoother than DQN's stepwise breakthrough. PPO plateaued earlier; matching DQN's growth requires training beyond 3M.

Statistical Significance of the Comparison

Evaluation protocol: each trained policy played **50 evaluation games** with **50 different environment seeds** (deterministic action selection, 3 lives per game).

Descriptive statistics

Metric	DQN	PPO
Mean eval score	21,916.00	10,312.00
Std deviation	5,429.60	5,108.45
Max observed	27,500	19,000
Mean difference (DQN − PPO)	+11,604	

Hypothesis tests

Test	DQN dist.	PPO dist.	p-value
Shapiro–Wilk (normality)	NOT normal	NOT normal	DQN 3.16e−9 / PPO 3.24e−8
Welch's t-test *(parametric)*	—	—	1.46e−18
Mann–Whitney U *(non-param.)*	—	—	**2.10e−11**

Interpretation

Non-normal distributions $\Rightarrow$ **Mann–Whitney U is the valid test**. $p = 2.10\text{e}{-11}$: probability the gap arose by chance is $\sim$1 in 50 billion. **DQN significantly outperforms PPO**.

Failure Analysis and Honest Limitations

Failure modes and experimental issues

- Reward hacking risks (identified during reward design — Slide 7): minion farming and crouching defense are theoretically incentivized by the sparse base reward; mitigated through custom shaping but not provably eliminated.
- Neither algorithm progressed beyond floor 1: agents repeatedly defeat the floor-1 boss but no transition to a higher floor was observed (the emulator does not appear to expose the floor-progression mechanic of the original cabinet — under investigation).
- PPO policy sensitivity: collapsed with DQN-default `step_penalty` (see Slide 14) — requires algorithm-aware reward design.

Algorithmic limitations

- **DQN:** "deadly triad" (function approximation + bootstrapping + off-policy) — no convergence guarantee under deep RL.
- **PPO:** on-policy = sample-inefficient; smoother but lower performance — consistent with literature on sparse-reward Atari.

Methodological limitations

- Single training seed (42) per algorithm; PPO's 8 envs each use different per-env seeds, providing some training-time variability.
- Generalization assessed via 50 different evaluation seeds.
- `repeat_action_probability = 0` — not standard ALE benchmark.
- PPO hyperparameters at SB3 defaults.
- Training stopped at 3M of planned 15M steps for time budget.
- No ablation studies.
- RAM used for reward computation only (Slide 13).

Takeaway

Our agents play KungFuMaster competently on floor 1 but do not solve the game. The DQN vs PPO comparison is valid *under our experimental conditions*; absolute scores reflect those conditions, not algorithmic ceilings.

Conclusion and References

Key findings

1. **DQN significantly outperforms PPO** on KungFuMaster under matched conditions. Mean: 21,916 vs 10,312; Mann–Whitney $p < 10^{-10}$.
2. **Algorithm-specific reward shaping is essential.** Same reward function, different parameter values: on-policy and off-policy methods do not interpret rewards identically.
3. **Game-specific preprocessing matters.** Custom knife dilation closed a perception gap.
4. Both algorithms reached competent floor-1 play; **neither progressed further**.

Future work

- Multi-seed training (beyond per-env PPO seeds) with confidence intervals on training variance.
- Extended budget (full 15M+ steps); Rainbow DQN; PPO + RND.
- Ablation studies on reward components; investigation of floor-progression mechanics in the ALE emulator.

Code and full training results

`https://github.com/AndreaZanin02/KungFuMaster`

References

1. Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning.* Nature 518(7540), 529–533.
2. Schulman, J., et al. (2017). *Proximal policy optimization algorithms.* arXiv:1707.06347.
3. van Hasselt, H., Guez, A., Silver, D. (2016). *Deep RL with double Q-learning.* AAAI.
4. Schulman, J., et al. (2016). *High-dimensional continuous control using GAE.* ICLR.
5. Bellemare, M.G., et al. (2013). *The arcade learning environment.* JAIR 47, 253–279.
6. Raffin, A., et al. (2021). *Stable-Baselines3: Reliable RL implementations.* JMLR 22(268).
7. Sutton, R.S., Barto, A.G. (2018). *Reinforcement Learning: An Introduction.* MIT Press.

Thank you. Questions?